Saveetha Institute of Medical and Technical Sciences

CSA02 – ASSIGNMENT 2

Grama Jyothi Rural Electricity Co-operative

Student Details	Information
Name	Hannah Fathima A
Register Number	1920260075
Programme	C Programming CSA0209
Submission Date	27 October 2026
GitHub Repository	**`https://github.com/hannahfathima360-ui/CSA02-1920260075-Assignment1`**
Nature of Assignment	Individual
Compiler & Version	GCC 13.2.0 (Ubuntu 22.04 LTS, x86_64, LP64 model)

1. Problem Analysis and Design

The program must calculate the number of units consumed from two meter readings, apply a tiered tariff, add a fixed charge, handle a 16-bit meter rollover, reject invalid readings, and print the total to the nearest paisa. The tariff must be defined once so a revision cannot be missed in multiple places.

Assumptions used because the question does not specify all operational details:

- The meter is 16-bit, so valid readings are 0 through 65535 (modulus 65536).
- If the current reading is less than the previous reading, exactly one rollover occurred between readings.
- The first 100 units are free; units 101–200 cost ₹2.25 per unit; units above 200 cost ₹4.50 per unit.
- The fixed charge is assumed to be ₹50.00. Replace FIXED_CHARGE_PAISE if the faculty specifies another amount.
- The tariff is progressive: each rate applies only to the units in its slab, not to all units consumed.
- All currency is stored as integer paise. One rupee equals 100 paise.

2. Program Structure

The solution separates declarations and implementation. billing.h contains central constants, the Bill structure, and function prototypes. billing.c contains reusable calculation and printing functions. main.c handles user input and calls the functions. This separation supports reuse and makes later tariff changes easier.

Code compilation note: billing.h, billing.c, and main.c form one program and must be compiled together using the command given below the main.c listing. format_demo.c and type_demo.c are separate demonstration programs; do not compile all .c files into one executable because each contains its own main() function.

File: billing.h

```
#ifndef BILLING_H
#define BILLING_H

#include <stdint.h>

/* Central tariff definitions. All money values are in paise. */
#define METER_MODULUS 65536U
#define FREE_UNITS 100U
#define SECOND_SLAB_LIMIT 200U
#define SECOND_SLAB_RATE_PAISE 225LL
#define HIGH_SLAB_RATE_PAISE 450LL
#define FIXED_CHARGE_PAISE 5000LL  /* Assumption: Rs 50.00 */

typedef struct {
    uint32_t previous_reading;
    uint32_t current_reading;
    uint32_t units;
    int64_t energy_charge_paise;
    int64_t fixed_charge_paise;
    int64_t total_paise;
} Bill;

int calculate_units(uint32_t previous, uint32_t current, uint32_t *units_out);
int64_t calculate_energy_charge(uint32_t units);
int calculate_bill(uint32_t previous, uint32_t current, Bill *bill_out);
void print_bill(const Bill *bill);

#endif
```

File: billing.c

```
#include <stdio.h>
#include "billing.h"

/* Return 1 on success, 0 for an invalid reading or NULL output pointer. */
int calculate_units(uint32_t previous, uint32_t current, uint32_t *units_out)
{
    if (units_out == NULL ||
        previous >= METER_MODULUS ||
        current >= METER_MODULUS) {
        return 0;
    }

    if (current >= previous) {
        *units_out = current - previous;
    } else {
        /* One 16-bit rollover: 65535 -> 0 means 1 unit. */
        *units_out = (METER_MODULUS - previous) + current;
    }

#ifdef DEBUG
    fprintf(stderr, "[DEBUG] previous=%u current=%u units=%u\n",
            (unsigned)previous, (unsigned)current, (unsigned)*units_out);
#endif
    return 1;
}

int64_t calculate_energy_charge(uint32_t units)
{
    uint32_t second_slab_units = 0;
    uint32_t high_slab_units = 0;

    if (units > FREE_UNITS) {
        second_slab_units = (units < SECOND_SLAB_LIMIT)
            ? units - FREE_UNITS
            : SECOND_SLAB_LIMIT - FREE_UNITS;
    }

    if (units > SECOND_SLAB_LIMIT) {
        high_slab_units = units - SECOND_SLAB_LIMIT;
    }

    int64_t charge =
        (int64_t)second_slab_units * SECOND_SLAB_RATE_PAISE +
        (int64_t)high_slab_units * HIGH_SLAB_RATE_PAISE;

#ifdef DEBUG
    fprintf(stderr, "[DEBUG] second_slab_units=%u high_slab_units=%u charge_paise=%lld\n",
            (unsigned)second_slab_units, (unsigned)high_slab_units,
            (long long)charge);
#endif
    return charge;
}

int calculate_bill(uint32_t previous, uint32_t current, Bill *bill_out)
{
    uint32_t units = 0;

    if (bill_out == NULL ||
        !calculate_units(previous, current, &units)) {
        return 0;
    }

    bill_out->previous_reading = previous;
    bill_out->current_reading = current;
    bill_out->units = units;
    bill_out->energy_charge_paise = calculate_energy_charge(units);
    bill_out->fixed_charge_paise = FIXED_CHARGE_PAISE;
    bill_out->total_paise =
        bill_out->energy_charge_paise + bill_out->fixed_charge_paise;

    return 1;
}
```

```
void print_bill(const Bill *bill)
{
    if (bill == NULL) {
        return;
    }

    printf("\n------ GRAMA JYOTHI ELECTRICITY BILL ------\n");
    printf("Previous reading : %u\n", (unsigned)bill->previous_reading);
    printf("Current reading  : %u\n", (unsigned)bill->current_reading);
    printf("Units consumed   : %u\n", (unsigned)bill->units);
    printf("Energy charge    : Rs %lld.%02lld\n",
           (long long)(bill->energy_charge_paise / 100),
           (long long)(bill->energy_charge_paise % 100));
    printf("Fixed charge     : Rs %lld.%02lld\n",
           (long long)(bill->fixed_charge_paise / 100),
           (long long)(bill->fixed_charge_paise % 100));
    printf("TOTAL BILL       : Rs %lld.%02lld\n",
           (long long)(bill->total_paise / 100),
           (long long)(bill->total_paise % 100));
    printf("-----------------------------------------\n");
}

#include <stdint.h>

#define METER_MODULUS 65536U
#define FREE_UNITS 100U
#define SECOND_SLAB_LIMIT 200U

#define SECOND_SLAB_RATE_PAISE 225
#define HIGH_SLAB_RATE_PAISE 450
#define FIXED_CHARGE_PAISE 5000

typedef struct {
    uint32_t previous_reading;
    uint32_t current_reading;
    uint32_t units;
    int64_t energy_charge_paise;
    int64_t fixed_charge_paise;
    int64_t total_paise;
} Bill;


/* Calculate units consumed, including one meter rollover. */
int calculate_units(uint32_t previous, uint32_t current,
                    uint32_t *units_out)
{
    if (units_out == NULL ||
        previous >= METER_MODULUS ||
        current >= METER_MODULUS) {
        return 0;
    }

    if (current >= previous) {
        *units_out = current - previous;
    } else {
        /* One rollover: 65535 -> 0 means 1 unit consumed. */
        *units_out = (METER_MODULUS - previous) + current;
    }

    return 1;
}


/* Calculate energy charges using the slab tariff. */
```

```
int64_t calculate_energy_charge(uint32_t units)
{
    uint32_t second_slab_units = 0;
    uint32_t high_slab_units = 0;

    if (units > FREE_UNITS) {
        if (units < SECOND_SLAB_LIMIT) {
            second_slab_units = units - FREE_UNITS;
        } else {
            second_slab_units = SECOND_SLAB_LIMIT - FREE_UNITS;
        }
    }

    if (units > SECOND_SLAB_LIMIT) {
        high_slab_units = units - SECOND_SLAB_LIMIT;
    }

    return (int64_t)second_slab_units * SECOND_SLAB_RATE_PAISE
         + (int64_t)high_slab_units * HIGH_SLAB_RATE_PAISE;
}


/* Calculate the complete electricity bill. */
int calculate_bill(uint32_t previous, uint32_t current, Bill *bill_out)
{
    uint32_t units = 0;

    if (bill_out == NULL ||
        !calculate_units(previous, current, &units)) {
        return 0;
    }

    bill_out->previous_reading = previous;
    bill_out->current_reading = current;
    bill_out->units = units;

    bill_out->energy_charge_paise = calculate_energy_charge(units);
    bill_out->fixed_charge_paise = FIXED_CHARGE_PAISE;

    bill_out->total_paise = bill_out->energy_charge_paise
                          + bill_out->fixed_charge_paise;

    return 1;
}


/* Display the electricity bill. */
void print_bill(const Bill *bill)
{
    if (bill == NULL) {
        return;
    }

    printf("\n------ GRAMA JYOTHI ELECTRICITY BILL ------\n");
    printf("Previous reading : %u\n",
           (unsigned)bill->previous_reading);
    printf("Current reading  : %u\n",
           (unsigned)bill->current_reading);
    printf("Units consumed   : %u\n",
           (unsigned)bill->units);
```

```
    printf("Energy charge    : Rs %lld.%02lld\n",
           (long long)(bill->energy_charge_paise / 100),
           (long long)(bill->energy_charge_paise % 100));

    printf("Fixed charge     : Rs %lld.%02lld\n",
           (long long)(bill->fixed_charge_paise / 100),
           (long long)(bill->fixed_charge_paise % 100));

    printf("TOTAL BILL       : Rs %lld.%02lld\n",
           (long long)(bill->total_paise / 100),
           (long long)(bill->total_paise % 100));

    printf("-----------------------------------------\n");
}


/* Main function */
int main(void)
{
    uint32_t previous, current;
    Bill bill;

    printf("GRAMa JYOTHI ELECTRICITY BILLING SYSTEM\n");

    printf("Enter previous meter reading (0-65535): ");
    if (scanf("%u", &previous) != 1) {
        printf("Error: Invalid previous reading.\n");
        return 1;
    }

    printf("Enter current meter reading (0-65535): ");
    if (scanf("%u", &current) != 1) {
        printf("Error: Invalid current reading.\n");
        return 1;
    }

    if (!calculate_bill(previous, current, &bill)) {
        printf("Error: Meter readings must be between 0 and 65535.\n");
        return 1;
    }

    print_bill(&bill);

    return 0;
}
```

```c

#include <stdio.h>
#include <stdint.h>

#define METER_MODULUS 65536U
#define FREE_UNITS 100U
#define SECOND_SLAB_LIMIT 200U

#define SECOND_SLAB_RATE_PAISE 225
#define HIGH_SLAB_RATE_PAISE 450
#define FIXED_CHARGE_PAISE 5000

typedef struct {
    uint32_t previous_reading;
    uint32_t current_reading;
    uint32_t units;
    int64_t energy_charge_paise;
    int64_t fixed_charge_paise;
    int64_t total_paise;
} Bill;


/* Calculate units consumed, including one
int calculate_units(uint32_t previous, ui
                    uint32_t *units_out)
{
    if (units_out == NULL ||
        previous >= METER_MODULUS ||
```

```
GRAMa JYOTHI ELECTRICITY BILLING SYSTEM
Enter previous meter reading (0-65535):
```

File: main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <ctype.h>
#include "billing.h"

/* Read one non-negative integer line, rejecting negatives and trailing junk. */
static int read_reading(const char *prompt, uint32_t *value_out)
{
    char buffer[128];
    char *end;
    unsigned long value;
    size_t i = 0;

    printf("%s", prompt);
    if (fgets(buffer, sizeof buffer, stdin) == NULL) {
        return 0;
    }

    while (isspace((unsigned char)buffer[i])) {
        i++;
    }
    if (buffer[i] == '-' || buffer[i] == '\0' ||
        buffer[i] == '\n') {
        return 0;
    }

    errno = 0;
    value = strtoul(buffer + i, &end, 10);
    if (errno != 0 || end == buffer + i || value >= METER_MODULUS) {
        return 0;
    }

    while (isspace((unsigned char)*end)) {
        end++;
    }
    if (*end != '\0') {
        return 0;
    }

    *value_out = (uint32_t)value;
    return 1;
}

int main(void)
{
    uint32_t previous;
    uint32_t current;
    Bill bill;

    puts("Grama Jyothi Electricity Billing");

    if (!read_reading("Enter previous meter reading (0-65535): ", &previous)) {
        fprintf(stderr, "Error: enter a whole-number reading from 0 to 65535.\n");
        return 1;
    }

    if (!read_reading("Enter current meter reading (0-65535): ", &current)) {
        fprintf(stderr, "Error: enter a whole-number reading from 0 to 65535.\n");
        return 1;
    }

    if (!calculate_bill(previous, current, &bill)) {
        fprintf(stderr, "Error: could not calculate the bill.\\n");
```

```
        return 1;
    }

    print_bill(&bill);
    return 0;
}
```

```c
int main(void)
{
    uint32_t previous;
    uint32_t current;
    Bill bill;

    puts("Grama Jyothi Electricity Billing");

    if (!read_reading("Enter previous meter reading (0-65535): ", &previous)) {
        fprintf(stderr, "Error: enter a whole-number reading from 0 to 65535.\n");
        return 1;
    }

    if (!read_reading("Enter current meter reading (0-65535): ", &current)) {
        fprintf(stderr, "Error: enter a whole-number reading from 0 to 65535.\n");
        return 1;
    }

    if (!calculate_bill(previous, current, &bill)) {
        fprintf(stderr, "Error: could not calculate the bill.\\n");
        return 1;
    }

    print_bill(&bill);
    return 0;
}
```

3. Task 1 – Header Files and Conditional Compilation (3 marks)

Include guards

The pattern #ifndef BILLING_H / #define BILLING_H / #endif ensures that the contents of billing.h are processed only once per translation unit. If the same header is included more than once, BILLING_H is already defined and the declarations are skipped. This prevents duplicate declaration problems.

Conditional compilation and DEBUG

The lines enclosed by #ifdef DEBUG are compiled only when DEBUG is defined. Build normally with `gcc -std=c11 -Wall -Wextra -Wpedantic main.c billing.c -o billing`; debug build with `gcc -std=c11 -Wall -Wextra -Wpedantic -DDEBUG main.c billing.c -o billing_debug`. In the debug build, diagnostic messages show readings, units, slab units and energy charge on stderr. In the normal build, those statements are removed by the preprocessor.

<file> versus "file"

`#include <stdio.h>` is conventionally used for system/library headers and searches implementation-defined system include paths. `#include "billing.h"` is used for a project header; the implementation first searches the directory associated with the source file (and then configured include paths). Exact search details are compiler-dependent.

Alternative: run-time debug variable

Alternative example: `if (debug) fprintf(stderr, "units=%u\\n", units);`. The executable retains the conditional branch and usually the format string and diagnostic code even if debug is false at run time. Each reached diagnostic site performs a Boolean test; its asymptotic cost is O(1) per site and O(k) for k sites. With #ifdef DEBUG and DEBUG undefined, debug statements and their strings are omitted, so there is no run-time test and less executable code/data. If compiled with -DDEBUG, the debug code is present and runs. In Big-O terms both approaches have O(1) overhead per check, but conditional compilation can remove the overhead entirely from a non-debug build. Actual byte sizes depend on compiler and optimisation flags.

4. Task 2 – Read and Print Data Correctly (4 marks)

Format specifier reference

Type / use	printf	scanf
int	%d	%d
unsigned int	%u	%u
long	%ld	%ld
long long	%lld	%lld
float	%f	%f
double	%f (or %lf)	%lf
char	%c	%c
string (char array)	%s	%s

Important nuance: for printf, a float argument is promoted to double, so %f prints a double. For scanf, %f expects a float* and %lf expects a double*. Use matching types. A mismatch in a variadic function can cause undefined behaviour; it is not merely a reliably predictable wrong number.

Input validation

scanf returns the number of input items successfully assigned. Since each reading is one conversion, `scanf("%u", &reading) != 1` means conversion failed. The supplied main.c checks both calls and exits with an error. A production-grade parser should also reject negative text for an unsigned reading and detect trailing junk; fgets plus strtoul can provide stricter validation.

Printing money

If amount_paise is a non-negative integer, rupees = amount_paise / 100 and paise = amount_paise % 100. Print with `printf("Rs %lld.%02lld", (long long)(amount_paise / 100), (long long)(amount_paise % 100));`. The `%02lld` ensures that 5 paise prints as `.05`, not `.5`.

Alternative: fgets plus strtol/sscanf

Read a complete line into a fixed-size buffer with fgets, then convert it with strtol (preferred for range/error checks) or sscanf. For a line of length L, scanning/conversion is O(L) time. Buffer space is O(B), where B is the chosen maximum line length; the program can detect overlong lines and drain the remainder. Direct scanf is also usually O(L) for the characters consumed, but invalid input can remain unread and cause repeated failures unless carefully handled. Line-based input makes it easier to reject a whole malformed line and recover for the next prompt. sscanf alone does not automatically provide robust range checking; strtol with end-pointer and errno checks is safer.

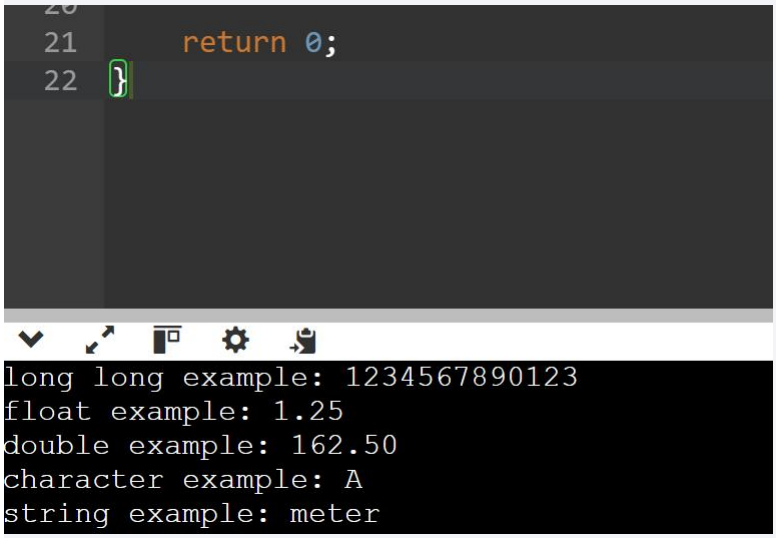
Format-specifier demonstration

```c
#include <stdio.h>

int main(void)
{
    long value = 123456789L;
    long long large_value = 1234567890123LL;
    float f = 1.25f;
    double amount = 162.50;
    char grade = 'A';
    char word[] = "meter";

    printf("int example: %d\n", 42);
    printf("unsigned int example: %u\n", 42U);
    printf("long example: %ld\n", value);
    printf("long long example: %lld\n", large_value);
    printf("float example: %.2f\n", (double)f);
    printf("double example: %.2f\n", amount);
    printf("character example: %c\n", grade);
    printf("string example: %s\n", word);

    return 0;
}
```



5. Task 3 – Type Conversion and Precision (4 marks)

Conversion points in the billing program

- scanf("%u", &previous_input) reads unsigned int; the value is explicitly cast to uint32_t when passed to calculate_bill.
- Readings and unit counts use uint32_t. Subtraction and addition are unsigned arithmetic; the rollover formula uses the 65536 modulus.
- Slab counts are uint32_t, while rates are long long constants. Each slab count is explicitly converted to int64_t before multiplication.
- Energy and fixed charges use int64_t paise. The total adds two int64_t values, avoiding a narrow int total.
- When printing, values are explicitly cast to long long to match the %lld conversion specifier.
- The currency print divides paise by 100 and uses remainder 100. Because the amount is stored in paise, no fractional currency arithmetic is needed.

Three or more surprising expressions

```
Expression: 199 / 100
Output: 1
Reason: both operands are integers, so integer division discards the fractional part. Use 199.0 / 100 for 1.99.

Expression: 5 / 2
Output: 2
Reason: integer division truncates toward zero. 5.0 / 2 gives 2.50.

Expression: (int)2.99
Output: 2
Reason: converting a floating value to int truncates the fractional part; it does not round to nearest.

Expression: (int)3000000000L
Result: implementation-defined when the value cannot be represented by int.
Reason: int may be 32-bit and its maximum is commonly 2147483647. Narrowing can lose information; do not store large
readings/totals in int.

Expression: 0.1 + 0.2 == 0.3
Output is commonly 0 (false) with binary floating point.
Reason: many decimal fractions have no exact finite binary representation.
```

The cast of an out-of-range long to int is implementation-defined in C (and must not be relied upon); it is not safe to claim a single universal output. A mismatched printf specifier such as using %d for a long is undefined behaviour, so its result is not portable.

Rounding rule

For this tariff, rates are exact whole paise per unit (225 and 450 paise), so multiplying whole units by integer paise-per-unit rates produces an exact integer amount. No rounding is needed for the energy charge. If a future tariff produces a fraction of one paisa, define a policy explicitly—for example, round half up to the nearest paisa—and apply it at a clearly specified stage. Do not mix ad hoc rounding rules.

Alternative: float calculations and cumulative error

A floating-point alternative would calculate rates in rupees using float, sum slab charges, and round once at the end, for example `roundf(total_rupees * 100.0f)`. However, float represents decimal rates approximately and errors can occur before the final rounding. Double is more precise than float, but integer paise is simpler when all tariff rates are exact paise.

If each bill has a small signed numeric error e_i before rounding, the absolute accumulated error across n bills is bounded by the sum of $|e_i|$; if the error magnitude is at most ε per bill, a worst-case bound is $n\varepsilon$. Errors may cancel in some datasets, so measured error is data- and compiler-dependent. Measure by comparing a float implementation against the integer-paise implementation for a large generated test set. Both approaches use O(1) time and O(1) space per bill; integer arithmetic generally has predictable exactness for these rates, while floating arithmetic can require rounding and can vary slightly by type/platform.

Demonstration program

```
#include <stdio.h>
#include <limits.h>

int main(void)
{
    int units = 199;
    long long large_value = 3000000000LL;

    printf("199 / 100 = %d\n", units / 100);
    printf("(int)2.99 = %d\n", (int)2.99);
    printf("5 / 2 = %d\n", 5 / 2);
    printf("5.0 / 2 = %.2f\n", 5.0 / 2.0);
    printf("INT_MAX = %d\n", INT_MAX);

    /*
     * This conversion is implementation-defined if large_value is not
     * representable as int. It is shown only as a conversion example;
     * do not rely on a particular result.
     */
    printf("(int)large_value is implementation-defined when out of range: %d\n",
           (int)large_value);

    return 0;
}
```

6. Task 4 – Trace the Compilation Process (4 marks)

Commands and generated artifacts

Stage	Command	Output / meaning
Preprocessing	gcc -E main.c -o main.i	Expanded source: headers included, macros expanded, conditional-compilation branches selected.
Assembly generation	gcc -S main.c -o main.s	Assembly language text for the target architecture.
Object compilation	gcc -c main.c -o main.o	Relocatable object file; not yet a complete executable.
Linking	gcc main.o billing.o -o billing	Combines object files and required libraries into an executable.

For the multi-file solution, compile billing.c as well: `gcc -c billing.c -o billing.o`.

What preprocessing changes

The preprocessor expands #include directives by inserting header contents, replaces macros such as FREE_UNITS and FIXED_CHARGE_PAISE with their definitions, and selects conditional branches. In a normal build, statements inside #ifdef DEBUG are removed because DEBUG is not defined. In a build using -DDEBUG, they are retained.

Compile-time, link-time, and run-time errors

- Compile-time error example: remove a closing parenthesis or write an invalid token in billing.c; GCC reports a syntax error and does not produce the object file.
- Link-time error example: declare a function in the header and call it, but omit its definition/object file when linking; the linker reports an undefined reference to calculate_bill or another missing function.
- Run-time error example: enter a letter where a numeric reading is expected; the program detects scanf returning 0, prints an error and exits. A run-time error is an issue during execution, not necessarily a crash.

Warnings and fixes

Compile with `-Wall -Wextra -Wpedantic`. These options help reveal issues such as format-string/type mismatches, unused variables, missing declarations, and suspicious conversions. Resolve warnings rather than suppressing them: match each format specifier to its argument type, include the correct headers, use explicit casts only when justified, and remove unused variables. Warning text depends on GCC version and the exact code; capture your own terminal output as evidence.

Alternative: several source files

The project already uses separate main.c and billing.c files with the shared billing.h header. Compile each .c file into an object file, then link the objects. If only billing.c changes, recompile billing.c to billing.o and relink; main.c need not be recompiled unless its source or a dependency such as billing.h changes. With a single source file, changing any part usually requires recompiling the whole translation unit. For a large project, separate compilation can reduce incremental build time. The space cost is the sum of object-file sizes plus the final executable; object files contain machine code, symbols and relocation data. Exact size depends on the compiler, architecture, optimisation and debug options.

7. Sample Test Cases and Expected Bills

Previous	Current	Units	Energy charge	Total with ₹50 fixed charge
1000	1100	100	₹0.00	₹50.00
1000	1200	200	₹225.00	₹275.00
1000	1300	300	₹675.00	₹725.00
65535	0	1	₹0.00	₹50.00
65530	5	11	₹0.00	₹50.00

For 300 units, the first 100 cost ₹0, the next 100 cost 100 × ₹2.25 = ₹225, and the final 100 cost 100 × ₹4.50 = ₹450. Energy charge = ₹675; adding the assumed ₹50 fixed charge gives ₹725. For a rollover from 65535 to 0, units = (65536 − 65535) + 0 = 1.

8. Smart-Meter Design Challenge: 15-Minute Readings and Time-of-Day Tariff

Decision: modify and extend the design rather than retain it unchanged. The integer-paise principle and centralised tariff configuration should be retained, but the data model and billing logic must be redesigned for interval records and time-dependent rates.

- Store each interval with meter ID, timestamp/time zone, start and end readings (or validated interval consumption), quality/status flags, and tariff period.
- Represent time-of-day tariff periods as configuration data with effective dates and rates rather than scattered constants. Keep rates in integer paise per kWh or use a documented scaled integer if sub-paise rates are legally required.
- Calculate energy separately for each interval and apply the tariff valid at that interval. A monthly bill is the sum of interval charges plus fixed charges and any other approved adjustments.
- Handle missing, duplicated, out-of-order, implausible, and rollover readings. Add idempotent processing so re-importing the same interval does not double-bill.
- Use 64-bit integers for cumulative paise and sufficiently wide counters; validate maximum consumption and overflow boundaries.
- Add tests for tariff changes mid-month, daylight-saving/time-zone boundaries where relevant, meter replacement, rollover, missing intervals, and exact paise totals.
- At district scale, process households in batches, use a database or durable event store, index by meter ID and timestamp, and log tariff-version IDs for auditability.
- Keep an audit trail of raw readings, corrections, tariff version, intermediate calculations, and final bill so disputes can be reproduced.

Complexity: for N interval records, a straightforward single pass performs O(N) time with O(1) working memory per record; storing all raw intervals requires O(N) storage. For monthly aggregation by household, maintain one running total per household (O(H) memory for H households) or aggregate in a database. At 15-minute intervals, a household has 96 intervals per day, or about 2,880 in a 30-day month, so 800,000 households could generate about 2.304 billion interval records per 30-day month. This scale justifies batch/stream processing and database design rather than loading all interval data into memory at once.

9. Build, Run and Evidence Checklist

☐ Create repository: CSA02-<RegisterNumber>-Assignment1.

☐ Add billing.h, billing.c, main.c, type_demo.c, format_demo.c and README.md.

☐ Run `gcc --version` and record the exact compiler version.

☐ Compile and run normal and DEBUG builds; save terminal screenshots.

☐ Run the five sample test cases and capture outputs.

☐ Test invalid text and out-of-range readings.

☐ Run the four GCC compilation-stage commands and save evidence of the generated files.

☐ Make progressive commits—at least one for each task—and share the repository with faculty.

☐ Paste the working GitHub URL on the cover page and submission field.

10. GitHub Commit Plan (example)

- `git add billing.h billing.c main.c && git commit -m "Task 1: add header and DEBUG mode"`
- `git add format_demo.c README.md && git commit -m "Task 2: validate input and document formats"`
- `git add type_demo.c && git commit -m "Task 3: demonstrate conversion and precision"`
- `git add . && git commit -m "Task 4: document compilation stages and evidence"`

Before submitting, replace the name/register-number placeholders, record your own GCC version, run the programs on your computer, capture your own screenshots, and paste your actual GitHub link. Do not claim measurements or screenshots you have not personally produced.

Compile the billing program from the folder containing all three files with: gcc -std=c11 -Wall -Wextra -Wpedantic main.c billing.c -o billing. Run with ./billing (Linux/macOS) or billing.exe (Windows). Compile format_demo.c and type_demo.c separately because each has its own main() function. The fixed charge of ₹50 is an assumption and should be changed if the instructor specifies a different value.